Copilot

Certainly! Let's optimize the Fibonacci sequence calculation using dynamic programming. The recursive approach you provided has exponential time complexity, which can be quite slow for large values of `n`. Here are a few more efficient approaches:

1. **Dynamic Programming (DP) Approach**:
   - Initialize an array `fib` with the first two Fibonacci numbers: `fib = [0, 1]`.
   - Then, iteratively build the sequence from the third number onward:

     ```
     for i in range(2, n + 1):
         fib.append(fib[i - 1] + fib[i - 2])
     ```

   - The result will be `fib[n]`.
   - This approach runs in linear time complexity: $$O(n)$$.

2. **Space-Optimized DP Approach**:
   - Instead of storing the entire sequence, keep track of only the last two Fibonacci numbers (`a` and `b`).
   - Initialize `a = 0` and `b = 1`.
   - Update them iteratively:

     ```
     for i in range(1, n):
         c = a + b
         a, b = b, c
     ```

   - The final result will be `b`.
   - This approach also runs in linear time complexity: $$O(n)$$.

3. **Using `functools.lru_cache`**:
   - Decorate a recursive function with `@lru_cache(None)` to cache results.
   - Here's an example:

     ```
     from functools import lru_cache

     @lru_cache(None)
     def fibonacci(num: int) -> int:
         if num < 2:
             return num
         return fibonacci(num - 1) + fibonacci(num - 2)

     print(fibonacci(9))  # Output: 34
     ```

   - This approach also runs in linear time complexity: $$O(n)$$.

Choose the approach that best suits your needs based on memory constraints and performance requirements. 😊